	r=3	r=5	r=7	r=9	r=11
Input_1.txt	.002 Seconds	.001 Seconds	.001 Seconds	.004 Seconds	.001 Seconds
Input_2.txt	.005 Seconds	.003 Seconds	.003 Seconds	.008 Seconds	.003 Seconds
Input_3.txt	.032 Seconds	.022 Seconds	.021 Seconds	.029 Seconds	.019 Seconds
Input_4.txt	.287 Seconds	.500 Seconds	.187 Seconds	.200 Seconds	.176 seconds
Input_5.txt	2.887 Seconds	2.086 Seconds	1.905 Seconds	1.855 Seconds	1.800 seconds

In the median of medians algorithm, when r was equal to 11 the program performed best. When looking at the recurrence of the algorithm for each r value we get a trend where the recurrence shrinks faster leading to the solution being found quicker. All of these were calculated by finding the worst case possible with calculations for r=3 and r=5 in part 2 and 3 of this project.

r	Recurrence Function	Amount of List being passed each stage
3	$T(n) \leq T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n)$	$\frac{1}{3} + \frac{2}{3} = 1$
5	$T(n) \leq T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n)$	$\frac{1}{5} + \frac{7}{10} = .9$
7	$T(n) \leq T(\frac{n}{7}) + T(\frac{5n}{7}) + O(n)$	$\frac{1}{7} + \frac{5}{7} = .857$
9	$T(n) \leq T(\frac{n}{9}) + T(\frac{13n}{18}) + O(n)$	$\frac{1}{9} + \frac{13}{18} = .833$
11	$T(n) \leq T(\frac{n}{11}) + T(\frac{8n}{11}) + O(n)$	$\frac{1}{11} + \frac{8}{11} = .818$

Looking at this table we see that as r increases the amount of the list being passed to the next recursive call, which is calculated by the addition of the two fractions from the recurrence functions, decreases. Typically when r is equal to 5 the best performance occurs due to the increasing overhead cost of sorting larger sub arrays but possibly due to modern CPU architecture with being able to cache larger amounts of data or that the amount of new vectors that need to be allocated each recursive call decreases results in a faster time than in theory.